

```

VecDoub arr(ndata);
for (j=0;j<ndata;j++) arr[j]=y[j]-b*x[j];
if ((ndata & 1) == 1) {
    a=select((ndata-1)>>1,arr);
} else {
    j=ndata >> 1;
    a=0.5*(select(j-1,arr)+select(j,arr));
}
abdev=0.0;
for (j=0;j<ndata;j++) {
    d=y[j]-(b*x[j]+a);
    abdev += abs(d);
    if (y[j] != 0.0) d /= abs(y[j]);
    if (abs(d) > EPS) sum += (d >= 0.0 ? x[j] : -x[j]);
}
return sum;
}
};

```

### 15.7.4 Other Robust Techniques

Sometimes you may have a priori knowledge about the probable values and probable uncertainties of some parameters that you are trying to estimate from a data set. In such cases you may want to perform a fit that takes this advance information properly into account, neither completely freezing a parameter at a predetermined value (as in Fitlin §15.4) nor completely leaving it to be determined by the data set. The formalism for doing this is called “use of a priori covariances.”

A related problem occurs in signal processing and control theory, where it is sometimes desired to “track” (i.e., maintain an estimate of) a time-varying signal in the presence of noise. If the signal is known to be characterized by some number of parameters that vary only slowly, then the formalism of *Kalman filtering* tells how the incoming raw measurements of the signal should be processed to produce best parameter estimates as a function of time. For example, if the signal is a frequency-modulated sine wave, then the slowly varying parameter might be the instantaneous frequency. The Kalman filter for this case is called a *phase-locked loop* and is implemented in the circuitry of modern radio receivers [4,5].

#### CITED REFERENCES AND FURTHER READING:

- Huber, P.J. 1981, *Robust Statistics* (New York: Wiley).[1]  
 Maronna, R., Martin, D., and Yohai, V. 2006, *Robust Statistics: Theory and Methods* (Hoboken, NJ: Wiley).[2]  
 Launer, R.L., and Wilkinson, G.N. (eds.) 1979, *Robustness in Statistics* (New York: Academic Press).[3]  
 Sayed, A.H. 2003, *Fundamentals of Adaptive Filtering* (New York: Wiley-IEEE).[4]  
 Harvey, A.C. 1989, *Forecasting, Structural Time Series Models and the Kalman Filter* (Cambridge, UK: Cambridge University Press).[5]

## 15.8 Markov Chain Monte Carlo

In this section and the next we redress somewhat the imbalance, at this point, between frequentist and Bayesian methods of modeling. Like Monte Carlo integra-

tion, *Markov chain Monte Carlo* or *MCMC* is a random sampling method. Unlike Monte Carlo integration, however, the goal of MCMC is not to sample a multidimensional region uniformly. Rather, the goal is to visit a point  $\mathbf{x}$  with a probability proportional to some given distribution function  $\pi(\mathbf{x})$ . The distribution  $\pi(\mathbf{x})$  is not quite a probability, because it is not necessarily normalized to have unity integral over the sampled region; but it is proportional to a probability.

Why would we want to sample a distribution in this way? The answer is that Bayesian methods, often implemented using MCMC, provide a powerful way of estimating the parameters of a model and their degree of uncertainty. A typical case is that there is a given set of data  $\mathbf{D}$ , and that we are able to calculate the probability of the data set *given* the values of the model parameters  $\mathbf{x}$ , that is,  $P(\mathbf{D}|\mathbf{x})$ . If we assume a prior  $P(\mathbf{x})$ , then Bayes' theorem says that the (posterior) probability of the model is proportional to  $\pi(\mathbf{x}) \equiv P(\mathbf{D}|\mathbf{x})P(\mathbf{x})$ , but with an unknown normalizing constant. Because of this unknown constant,  $\pi(\mathbf{x})$  is not a normalized probability density. But if we can sample from it, we can estimate any quantity of interest, for example its mean or variance. Indeed, we can readily recover a normalized probability density by observing how often we sample a given volume  $d\mathbf{x}$ . Often even more useful, we can observe the distribution of any single component or set of components of the vector  $\mathbf{x}$ , equivalent to *marginalizing* (i.e., integrating over) the other components.

We could in principle obtain all the same information by ordinary Monte Carlo integration over the region of interest, computing the value of  $\pi(\mathbf{x}_i)$  at every (uniformly) sampled point  $\mathbf{x}_i$ . The huge advantage of MCMC is that it “automatically” puts its sample points preferentially where  $\pi(\mathbf{x})$  is large (in fact, in direct proportion). In a high-dimensional space, or where  $\pi(\mathbf{x})$  is expensive to compute, this can be advantageous by many orders of magnitude.

Two insights, originally due to Metropolis and colleagues in the early 1950s, lead to feasible MCMC methods. The first is the idea that we should try to sample  $\pi(\mathbf{x})$  not via unrelated, independent points, but rather by a *Markov chain*, a sequence of points  $\mathbf{x}_0, \mathbf{x}_1, \mathbf{x}_2, \dots$  that, while locally correlated, can be shown to eventually visit every point  $\mathbf{x}$  in proportion to  $\pi(\mathbf{x})$ , the *ergodic* property. Here the word “Markov” means that each point  $\mathbf{x}_i$  is chosen from a distribution that depends only on the value of the immediately preceding point  $\mathbf{x}_{i-1}$ . In other words, the chain has memory extending only to one previous point and is completely defined by a transition probability function of two variables  $p(\mathbf{x}_i|\mathbf{x}_{i-1})$ , the probability with which  $\mathbf{x}_i$  is picked given a previous point  $\mathbf{x}_{i-1}$ .

The second insight is that if  $p(\mathbf{x}_i|\mathbf{x}_{i-1})$  is chosen to satisfy the *detailed balance equation*,

$$\pi(\mathbf{x}_1)p(\mathbf{x}_2|\mathbf{x}_1) = \pi(\mathbf{x}_2)p(\mathbf{x}_1|\mathbf{x}_2) \quad (15.8.1)$$

then (up to some technical conditions) the Markov chain will in fact sample  $\pi(\mathbf{x})$  ergodically. This amazing fact is worthy of some contemplation. Equation (15.8.1) expresses the idea of *physical equilibrium* in the reversible transition

$$\mathbf{x}_1 \longleftrightarrow \mathbf{x}_2 \quad (15.8.2)$$

That is, if  $\mathbf{x}_1$  and  $\mathbf{x}_2$  occur in proportion to  $\pi(\mathbf{x}_1)$  and  $\pi(\mathbf{x}_2)$ , respectively, then the overall transition rates in each direction, each the product of a population density and a transition probability, are the same. To see that this might have something to do with the Markov chain being ergodic, integrate both sides of equation (15.8.1) with

respect to  $\mathbf{x}_1$ :

$$\int p(\mathbf{x}_2|\mathbf{x}_1)\pi(\mathbf{x}_1) d\mathbf{x}_1 = \pi(\mathbf{x}_2) \int p(\mathbf{x}_1|\mathbf{x}_2) d\mathbf{x}_1 = \pi(\mathbf{x}_2) \quad (15.8.3)$$

The left-hand side of equation (15.8.3) is the probability of  $\mathbf{x}_2$ , computed by integrating over all possible values of  $\mathbf{x}_1$  with the corresponding transition probability. The right-hand side is seen to be the desired  $\pi(\mathbf{x}_2)$ . So equation (15.8.3) says that if  $\mathbf{x}_1$  is drawn from  $\pi$ , then *so is its successor* in the Markov chain,  $\mathbf{x}_2$ .

We also need to show that the equilibrium distribution is rapidly approached from any starting point  $\mathbf{x}_0$ . While the formal proof is beyond our scope, a heuristic proof is to recognize that, because of ergodicity, even very unlikely values  $\mathbf{x}_0$  will be visited by the equilibrium Markov chain once in a great while. Since the chain has no past memory, choosing any such point as a starting point  $\mathbf{x}_0$  is equivalent to just picking up the equilibrium distribution chain at that particular point in time, q.e.d. In practice we need to recognize that when we start from a very unlikely point, successor points will themselves be quite unlikely until we rejoin a more probable part of the distribution. There is thus a need to *burn-in* an MCMC chain by stepping through, and discarding, a certain number of points  $\mathbf{x}_i$ . Below, we discuss how to determine the length of the burn-in.

We can gain a better understanding the nature of the approach to  $\pi$  using concepts from §11.0 and (in the next chapter) §16.3. Heuristically, let us pretend that the states  $x_i$  are discrete. Then  $p(x_j|x_i) \equiv P_{ij}$  is a transition matrix satisfying equation (16.3.1). The discussion following equation (16.3.4) shows that the matrix  $\mathbf{P}^T$  must have at least one unity eigenvalue. In fact, the vector  $\boldsymbol{\pi}$  (the discrete form of the distribution  $\pi(\mathbf{x})$ ) is an eigenvector of  $\mathbf{P}^T$  with unity eigenvalue, by equation (15.8.3).

Can there be eigenvalues with magnitude greater than unity? No. Suppose to the contrary that  $\lambda > 1$  is the largest eigenvalue, with eigenvector  $\mathbf{v}$ . Then, repeatedly applying  $\mathbf{P}^T$ ,

$$\lim_{n \rightarrow \infty} (\mathbf{P}^T)^n \cdot \mathbf{v} = \lambda^n \mathbf{v} \rightarrow \infty \times \mathbf{v} \quad (15.8.4)$$

Any starting distribution that contains even a tiny piece of  $\mathbf{v}$  (always possible to arrange) will be driven to have values either  $< 0$  or  $> 1$ , which is impossible. Hence it must be that  $\lambda \leq 1$ .

From an arbitrary starting distribution  $\mathbf{u}$ , repeated steps of  $\mathbf{P}^T$  must thus converge to  $\boldsymbol{\pi}$  geometrically, with a rate that is asymptotically the magnitude of the second-largest eigenvalue, which will be  $< 1$  if  $\boldsymbol{\pi}$  is the unique equilibrium distribution. If the second eigenvalue is small, the distribution  $p(x_j|x_i)$  is said to be *rapidly mixing*.

Obviously missing from this discussion, and beyond our scope, is a discussion of degenerate eigenvalues (related to the question of uniqueness) and a continuous, rather than discrete, treatment. In practice, one rarely knows enough about  $\mathbf{P}$  to compute useful bounds on the second eigenvalue a priori.

### 15.8.1 Metropolis-Hastings Algorithm

Unless we can find a transition probability function  $p(\mathbf{x}_2|\mathbf{x}_1)$  that satisfies the detailed balance equation (15.8.1), we have no way to proceed. Luckily, Hastings [1], generalizing Metropolis' work, has given a very general prescription:

Pick a *proposal distribution*  $q(\mathbf{x}_2|\mathbf{x}_1)$ . This can be pretty much anything you want, as long as a succession of steps generated by it can, in principle, reach everywhere in the region of interest. For example,  $q(\mathbf{x}_2|\mathbf{x}_1)$  might be a multivariate normal distribution centered on  $\mathbf{x}_1$ .

Now, to generate a step starting at  $\mathbf{x}_1$ , first generate a *candidate point*  $\mathbf{x}_{2c}$  by drawing from the proposal distribution. Second, calculate an *acceptance probability*  $\alpha(\mathbf{x}_1, \mathbf{x}_{2c})$  by the formula

$$\alpha(\mathbf{x}_1, \mathbf{x}_{2c}) = \min \left( 1, \frac{\pi(\mathbf{x}_{2c}) q(\mathbf{x}_1 | \mathbf{x}_{2c})}{\pi(\mathbf{x}_1) q(\mathbf{x}_{2c} | \mathbf{x}_1)} \right) \quad (15.8.5)$$

Finally, with probability  $\alpha(\mathbf{x}_1, \mathbf{x}_{2c})$ , accept the candidate point and set  $\mathbf{x}_2 = \mathbf{x}_{2c}$ ; otherwise reject it and leave the point unchanged (that is,  $\mathbf{x}_2 = \mathbf{x}_1$ ). The net result of this process is a transition probability,

$$p(\mathbf{x}_2 | \mathbf{x}_1) = q(\mathbf{x}_2 | \mathbf{x}_1) \alpha(\mathbf{x}_1, \mathbf{x}_2), \quad (\mathbf{x}_2 \neq \mathbf{x}_1) \quad (15.8.6)$$

To see how this satisfies detailed balance, first multiply equation (15.8.5) by the denominator in the second argument of the min function. Then write down the identical equation, but exchange  $\mathbf{x}_1$  and  $\mathbf{x}_2$ . From these pieces, one writes,

$$\begin{aligned} \pi(\mathbf{x}_1) q(\mathbf{x}_2 | \mathbf{x}_1) \alpha(\mathbf{x}_1, \mathbf{x}_2) &= \min[\pi(\mathbf{x}_1) q(\mathbf{x}_2 | \mathbf{x}_1), \pi(\mathbf{x}_2) q(\mathbf{x}_1 | \mathbf{x}_2)] \\ &= \min[\pi(\mathbf{x}_2) q(\mathbf{x}_1 | \mathbf{x}_2), \pi(\mathbf{x}_1) q(\mathbf{x}_2 | \mathbf{x}_1)] \quad (15.8.7) \\ &= \pi(\mathbf{x}_2) q(\mathbf{x}_1 | \mathbf{x}_2) \alpha(\mathbf{x}_2, \mathbf{x}_1) \end{aligned}$$

which, using equation (15.8.6), can be seen to be exactly the detailed balance equation (15.8.1).

It is often possible to choose the proposal distribution  $q(\mathbf{x}_2 | \mathbf{x}_1)$  in such a way as to simplify equation (15.8.5). For example, if  $q(\mathbf{x}_2 | \mathbf{x}_1)$  depends only on the absolute difference  $|\mathbf{x}_1 - \mathbf{x}_2|$ , as in the case of a normal distribution with fixed covariance, then the ratio  $q(\mathbf{x}_1 | \mathbf{x}_{2c})/q(\mathbf{x}_{2c} | \mathbf{x}_1)$  is just 1. Another case that occurs frequently is when, for some component  $x$  of  $\mathbf{x}$ ,  $q(x_{2c} | x_1)$  is lognormally distributed with a mode at  $x_1$ . In that case the ratio for this component is  $x_{2c}/x_1$  (cf. equation 6.14.31).

## 15.8.2 Gibbs Sampler

An important special case of the Metropolis-Hastings algorithm is the *Gibbs sampler*. (Historically, the Gibbs sampler was developed independently of Metropolis-Hastings, see [2,5], but we discuss it here in a unified framework.) The Gibbs sampler is based on the fact that a multivariate distribution is uniquely determined by the set of all of its full conditional distributions; but if you don't know what this means, just read on anyway.

A *full conditional distribution* of  $\pi(\mathbf{x})$  is obtained by holding all of the components of  $\mathbf{x}$  constant *except one* (call it  $x$ ), and then sampling as a function of  $x$  alone. In other words, it is the distribution that you see when you “drill through”  $\pi(\mathbf{x})$  along a coordinate direction, and with fixed values of all the other coordinates. We'll denote a full conditional distribution by the notation  $\pi(x | \mathbf{x}^-)$ , where  $\mathbf{x}^-$  means “values of all the coordinates except one.” (To keep the notation readable, we are suppressing an index  $i$  that would tell which component of  $\mathbf{x}$  is  $x$ .)

Suppose that we construct a Metropolis-Hastings chain that allows only the one coordinate  $x$  to vary. Then equation (15.8.5) would look like this:

$$\alpha(x_1, x_{2c} | \mathbf{x}^-) = \min \left( 1, \frac{\pi(x_{2c} | \mathbf{x}^-) q(x_1 | x_{2c}, \mathbf{x}^-)}{\pi(x_1 | \mathbf{x}^-) q(x_{2c} | x_1, \mathbf{x}^-)} \right) \quad (15.8.8)$$

Now let's pick as our proposal distribution,

$$q(x_2|x_1, \mathbf{x}^-) = \pi(x_2|\mathbf{x}^-) \quad (15.8.9)$$

Look what happens: The second argument of the min function becomes 1, so the acceptance probability  $\alpha$  is also 1. In other words, if we propose a value  $x_2$  from the full conditional distribution  $\pi(x_2|\mathbf{x}^-)$ , we can always accept it. The advantage is obvious. The disadvantage is that the full conditional distribution *must* be properly normalized as a probability distribution — otherwise how could we use it as a transition probability? Thus, we will usually need to calculate (either analytically or by numerical integration) the normalizing constant

$$\int \pi(x|\mathbf{x}^-) dx \quad (15.8.10)$$

for every  $\mathbf{x}^-$  of interest, and we will need to have a practical algorithm for drawing  $x_2$  from the thus-normalized distribution. Note that these one-dimensional normalizing constants are *much* easier to compute than would be the multidimensional normalizing constant for the whole distribution  $\pi(\mathbf{x})$ .

The full Gibbs sampler operates as follows: Cycle through each component of  $\mathbf{x}$  in turn. (A fixed cyclical order is usually used, but choosing a component randomly each time is also fine.) For each component, hold all the other components fixed and draw a new value  $x$  from the full conditional distribution  $\pi(x|\mathbf{x}^-)$  of all possible values of that component. (This is where you might have to do a numerical integral at each step.) Set the component to the new value and go on to the next component.

You can see that the Gibbs sampler is “more global” than the regular Metropolis-Hastings algorithm. At each step, a component of  $\mathbf{x}$  gets reset to a value completely independent of its previous value (independent, at least, in the conditional distribution). If we tried to get behavior like this with regular Metropolis-Hastings, by proposing really big multivariate normal steps, say, we would get nowhere, since the steps would be almost always rejected!

On the other hand, the need to draw from a normalized conditional distribution can be a real killer in terms of computational workload. Gibbs sampling can be recommended enthusiastically when the components of  $\mathbf{x}$  have discrete, not continuous, values, and not too many possible values for each component. In that case the normalization is just a sum over not-too-many terms, and the Gibbs sampler can be very efficient. For the case of continuous variables, you are probably better off with regular Metropolis-Hastings, unless your particular problem admits to some fast, tricky way of getting the normalizations.

Don't confuse the Gibbs sampler with the tactic of doing regular Metropolis-Hastings steps along one component at a time. For the latter, we restrict the proposal distribution to proposing a change in a single component, either randomly chosen or else cycling through all the components in a regular order. This is sometimes useful if it lets us compute  $\pi(\mathbf{x})$  more efficiently (e.g., using saved pieces from the previous calculation on components that have not changed). What makes this *not* Gibbs is that we calculate an acceptance probability in the regular way, with equation (15.8.5) and the full distribution  $\pi(\mathbf{x})$ , which need not be normalized.

### 15.8.3 MCMC: A Worked Example

A number of practical details regarding MCMC are best discussed in the context of a worked example:

*At the beginning of an experiment, events occur Poisson randomly with a mean rate  $\lambda_1$ , but only every  $k_1$ th event is recorded. Then, at time  $t_c$ , the mean rate changes to  $\lambda_2$ , but now only every  $k_2$ th event is recorded. We are given the times  $t_0, \dots, t_{N-1}$  of the  $N$  recorded events. Oh, by the way, the values  $\lambda_1$ ,  $\lambda_2$ ,  $k_1$ ,  $k_2$ , and  $t_c$  are all unknown. We want to find them.*

Let's decompose the separate parts of the calculation into separate objects. First we need an object that represents the point  $\mathbf{x}$ . Although we've been discussing  $\mathbf{x}$  as if it were a vector, it can actually be a mixture of continuous, discrete, boolean, or any other kind of variable. In our example we have both continuous and discrete variables.

```
struct State {
    Doub lam1, lam2;       $\lambda_1$  and  $\lambda_2$ 
    Doub tc;               $t_c$ 
    Int k1, k2;            $k_1$  and  $k_2$ 
    Doub plog;            Set to  $\log P$  by Plog, below.

    State(Doub la1, Doub la2, Doub t, Int kk1, Int kk2) :
        lam1(la1), lam2(la2), tc(t), k1(kk1), k2(kk2) {}
    State() {}
};
```

mcmc.h

Worked MCMC example: Structure containing the components of  $\mathbf{x}$ .

The constructor is used to set initial values. (The plog variable is not part of  $\mathbf{x}$ , but it will be used later.)

Next, we need an object for calculating  $\pi(\mathbf{x}) = P(\mathbf{D}|\mathbf{x})$ , the probability of the data given the parameters. For our example, we need to use a couple of facts about Poisson processes: If a Poisson process has a rate  $\lambda$ , then the waiting time to the  $k$ th event is distributed as  $\text{Gamma}(k, \lambda)$ , that is,

$$p(\tau|k, \lambda) = \frac{\lambda^k}{(k-1)!} \tau^{k-1} e^{-\lambda\tau} \quad (15.8.11)$$

where  $\tau = t_{i+k} - t_i$ . (Compare equation 6.14.41, and also §7.3.10.) The exponential distribution is a special case with  $k = 1$ . Further, probabilities for non-overlapping intervals such as  $t_{i+k} - t_i$  and  $t_{i+2k} - t_{i+k}$  are independent. It follows that, for our example,

$$P(\mathbf{D}|\mathbf{x}) = \prod_{t_i \leq t_c} p(t_{i+1} - t_i | k_1, \lambda_1) \times \prod_{t_i > t_c} p(t_{i+1} - t_i | k_2, \lambda_2) \quad (15.8.12)$$

where  $p(\tau|k, \lambda)$  is as given in (15.8.11), and where  $t_i$  is now the  $i$ th *recorded* time. (In the words following equation 15.8.11,  $t_i$  was the  $i$ th event whether recorded or not.)

Actually, as the amount of data gets large,  $P(\mathbf{D}|\mathbf{x})$  is likely to over- or underflow, so it is best to calculate  $\log P$ . It is important to make this calculation as

efficient as possible, because it will be done at every step. Particularly important is to minimize the amount of looping over all the data points. In our example, if you take the logarithm of equations (15.8.11) and (15.8.12), you'll see that the individual  $t_i$ 's enter into  $\log P$  only as a sum of intervals and sum of log of intervals, less than and greater than  $t_c$ . An efficient way to proceed is thus to digest the data once and store two cumulative sums. Then, given a value  $t_c$ , we can find our place in the table of sums by bisection and read off the left and right sums directly. There is thus no loop over the data at all! Life is rarely so good, but when it is, then *carpe diem*. The resulting object looks like this:

```
mcmc.h struct Plog {
    Functor that calculates  $\log P$  of a State.
    VecDoub &dat;                                Bind to data vector.
    Int ndat;
    VecDoub stau, slogtau;

    Plog(VecDoub &data) : dat(data), ndat(data.size()),
        stau(ndat), slogtau(ndat) {
        Constructor. Digest the data vector for subsequent fast calculation of  $\log P$ . The data are
        assumed to be sorted in ascending order.
        Int i;
        stau[0] = slogtau[0] = 0.;
        for (i=1;i<ndat;i++) {
            stau[i] = dat[i]-dat[0];                Equal to sum of intervals.
            slogtau[i] = slogtau[i-1] + log(dat[i]-dat[i-1]);
        }
    }

    Doub operator() (State &s) {
        Return  $\log P$  of s, and also set s.plog.
        Int i,ilo,ihi,n1,n2;
        Doub st1,st2,stl1,stl2, ans;
        ilo = 0;
        ihi = ndat-1;
        while (ihi-ilo>1) {
            i = (ihi+ilo) >> 1;
            if (s.tc > dat[i]) ilo=i;
            else ihi=i;
        }
        n1 = ihi;
        n2 = ndat-1-ihi;
        st1 = stau[ihi];
        st2 = stau[ndat-1]-st1;
        stl1 = slogtau[ihi];
        stl2 = slogtau[ndat-1]-stl1;
        Equations (15.8.11) and (15.8.12):
        ans = n1*(s.k1*log(s.lam1)-factln(s.k1-1))+(s.k1-1)*stl1-s.lam1*st1;
        ans += n2*(s.k2*log(s.lam2)-factln(s.k2-1))+(s.k2-1)*stl2-s.lam2*st2;
        return (s.plog = ans);
    }
};
```

The Plog object is the only place that the data enter, and they enter only through the constructor. All other parts of the calculation see the data only through the calculation of  $\log P$ .

Next we come to the proposal generator, which we call Proposal. It doesn't have any contact with the data, or with  $\log P$ . All it needs to know about is the domain of  $\mathbf{x}$  (that is, State). It is worth thinking hard about the proposal gener-



ator. Although “almost any” generator will work in theory, a poor generator will take longer than the age of the universe to converge, while a good, *rapidly mixing* generator can go like lightning. This is where MCMC starts becoming an art.

Our example is designed to furnish an illustration of this in the interaction between the  $\lambda$  parameters and their corresponding  $k$ ’s. The mean rate of recorded counts is  $\lambda/k$ . Since  $\lambda$  is a continuous variable, we will be proposing relatively small changes in it at each step. Since  $k$  is discrete, there is no such thing as a small change, especially when  $k$  is small.

If we naively write a generator that proposes random independent changes in  $\lambda$  and  $k$ , then, after we have settled down to roughly the right value of  $\lambda/k$ , essentially all proposals for changing  $k$  will be rejected. The reason is that the acceptable step in  $\lambda$  required for a change in  $k$  from 1 to 2 (say) is so large (doubling  $\lambda$ ) that our generator will pick it only, say, every billion years! If we are not smart enough to recognize this problem ahead of time, we can find it experimentally by inspecting the Markov chain as it evolves and noting the proposals to change  $k$  are never accepted.

A solution in our case is to have two kinds of steps. The first changes  $\lambda$  (by a small amount) and keeps  $k$  fixed. The second changes  $k$  and  $\lambda$ , keeping  $\lambda/k$  fixed. We choose randomly between the two kinds of steps, mostly choosing the first kind.

The general issue here is what to do when  $\pi(\mathbf{x})$  defines some highly correlated directions among the components in  $\mathbf{x}$ . If you can recognize these directions, your proposal generator should, at least sometimes, generate proposals along them. Otherwise, it will have to propose very small steps, if they are ever to be accepted. In our example, this latter choice was made impossible by the discreteness in  $k$ , forcing us to diagnose and confront the issue directly. So, although `Proposal` doesn’t directly have to know about  $\log P$ , you may need a qualitative understanding of  $\log P$  when you design `Proposal`.

Since only `Proposal` knows the algorithm by which a proposal is generated, this object must also calculate, and return, the ratio  $q(\mathbf{x}_1|\mathbf{x}_{2c})/q(\mathbf{x}_{2c}|\mathbf{x}_1)$ , which is needed in equation (15.8.5). Here is an example that proposes small lognormal steps for the variables  $\lambda_1$ ,  $\lambda_2$ , and  $t_c$ , or else proposes incrementing  $k_1$  and  $k_2$  by 1, 0, or  $-1$ , with corresponding changes in the  $\lambda$ ’s as described above.

```
struct Proposal { mcmc.h
    Functor implementing the proposal distribution.
    Normaldev gau;
    Doub logstep;

    Proposal(Int ranseed, Doub lstep) : gau(0.,1.,ranseed), logstep(lstep) {}

    void operator() (const State &s1, State &s2, Doub &qratio) {
        Given state s1, set state s2 to a proposed candidate. Also set qratio to  $q(s1|s2)/q(s2|s1)$ .

        Doub r=gau.doub();
        if (r < 0.9) { Lognormal steps holding the  $k$ ’s constant.
            s2.lam1 = s1.lam1 * exp(logstep*gau.dev());
            s2.lam2 = s1.lam2 * exp(logstep*gau.dev());
            s2.tc = s1.tc * exp(logstep*gau.dev());
            s2.k1 = s1.k1;
            s2.k2 = s1.k2;
            qratio = (s2.lam1/s1.lam1)*(s2.lam2/s1.lam2)*(s2.tc/s1.tc);
            Factors for lognormal steps.
        } else { Steps that change  $k_1$  and/or  $k_2$ .
            r=gau.doub();
```



```

    if (s1.k1>1) {
        if (r<0.5) s2.k1 = s1.k1;
        else if (r<0.75) s2.k1 = s1.k1 + 1;
        else s2.k1 = s1.k1 - 1;
    } else {
        if (r<0.75) s2.k1 = s1.k1;
        else s2.k1 = s1.k1 + 1;
    }
    s2.lam1 = s2.k1*s1.lam1/s1.k1;
    r=gau.doub();
    if (s1.k2>1) {
        if (r<0.5) s2.k2 = s1.k2;
        else if (r<0.75) s2.k2 = s1.k2 + 1;
        else s2.k2 = s1.k2 - 1;
    } else {
        if (r<0.75) s2.k2 = s1.k2;
        else s2.k2 = s1.k2 + 1;
    }
    s2.lam2 = s2.k2*s1.lam2/s1.k2;
    s2.tc = s1.tc;
    qratio = 1.;
}
}
};

```

$k_1 = 1$  requires special treatment.

Now all the same for  $k_2$ .

(We use the convenient fact that since Normaldev is derived from Ran, it contains both normal and uniform random number generators.)

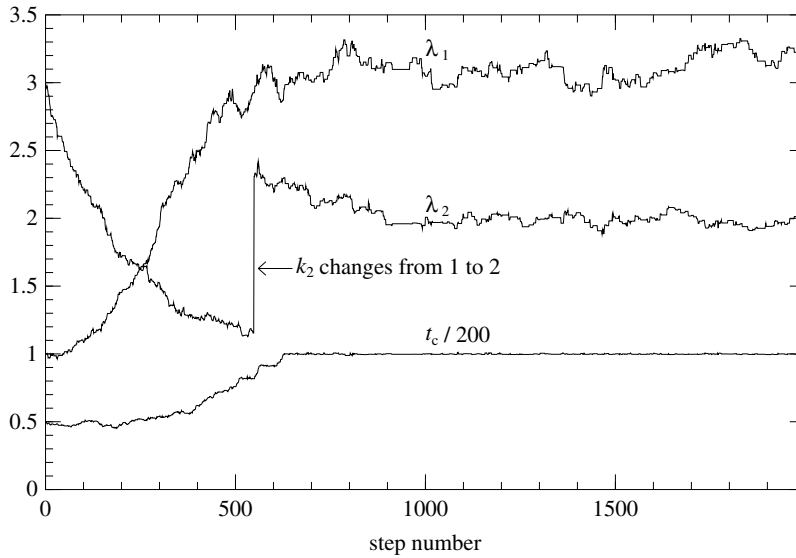
How shall we set logstep, the size of the proposed lognormal step? A rule of thumb for proposals like this with an adjustable scale is that the average acceptance probability ought to be roughly between 0.1 and 0.4. If it is very much smaller, then decrease the step size parameter; if it is much larger, then increase the step size parameter. In our example, the value  $\text{logstep} = 0.01$  (i.e., proposed changes on the order of  $\pm 1\%$ ) gives good results.

Finally, there is a function that takes a specified number of steps, implementing equation (15.8.5). This short piece of code is about the only “universal” part of MCMC; it has no persistent state and gets all the information it needs via the State, Plog, and Proposal structures. As we have seen, these are all problem-dependent and benefit from cleverness and special tricks.

```

mcmc.h  Doub mcmcstep(Int m, State &s, Plog &plog, Proposal &propose) {
    Take m MCMC steps, starting with (and updating) s.
    State sprop;                               Storage for candidate.
    Doub qratio, alph, ran;
    Int accept=0;
    plog(s);
    for (Int i=0; i<m; i++) {                   Loop over steps.
        propose(s, sprop, qratio);
        alph = min(1., qratio*exp(plog(sprop)-s.plog()));   Equation (15.8.5).
        ran = propose.gau.doub();
        if (ran < alph) {                       Accept the candidate.
            s = sprop;
            plog(s);
            accept++;
        }
    }
    return accept/Doub(m);
}

```



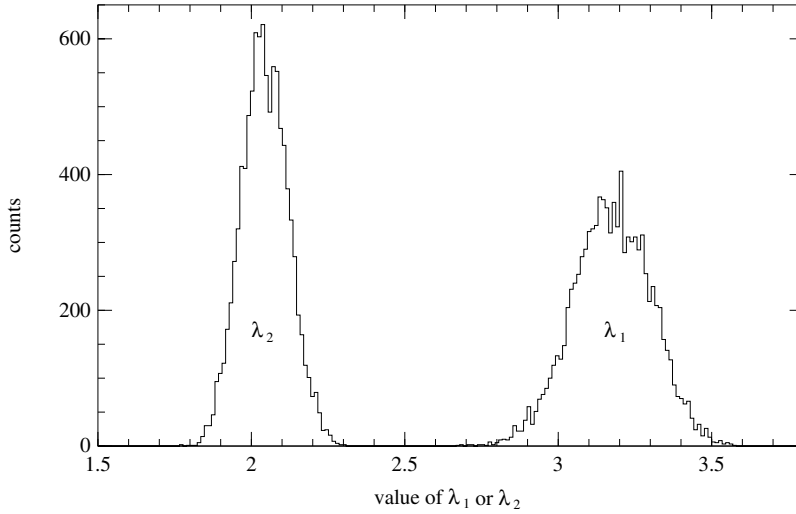
**Figure 15.8.1.** Evolution of model parameters  $\lambda_1$ ,  $\lambda_2$ , and  $t_c$  as a function of Markov chain Monte Carlo step. In this example, the burn-in time is seen to be  $\sim 1000$  steps, after which the Markov chain explores the equilibrium distribution.

Let's try it all out. We'll assume  $N = 1000$  data points  $t_i$  and start  $\mathbf{x}$  with the values  $\lambda_1 = 1$ ,  $\lambda_2 = 3$ ,  $t_c = 100$ , and  $k_1 = k_2 = 1$ . (Secretly, we know that the data were generated using actual values 3, 2, 200, 1, 2, respectively.) The random seed is 10102, and the lognormal stepsize is 0.01. We'll take 1000 steps of burn-in, and thereafter store values after every 10 steps. Driver code in `main` for this run is

```
VecDoub times(1000);
...
State s(1.,3.,100.,1,1);
Plog plog(times);
Proposal propose(10102,.01);
for (i=0;i<1000;i++) accept = mcmcstep(1,s,plog,propose);  Burn-in.
for (i=0;i<10000;i++) {                                     Production.
    accept = mcmcstep(10,s,plog,propose);
    ...             Save values, increment averages, etc., here.
}
```

Figure 15.8.1 shows the evolution of the parameters  $\lambda_1$ ,  $\lambda_2$  and  $t_c$ . During burn-in, you can see the parameters heading toward equilibrium, mostly monotonically, but with the exception of  $\lambda_2$ , which goes rapidly toward the value 1, with the value  $k = 1$ . These values indeed replicate the mean rate of the recorded data. Only when it is near convergence (around step 560), does the model discover that the  $t_i$ 's greater than  $t_c$  don't actually fit an exponential distribution ( $k_2 = 1$ ) but *do* fit a gamma distribution with the same mean rate, but with  $k_2 = 2$  (the correct answer). Had we not provided `Proposal` with a step that tests for this, we would likely have converged to a wrong answer. More precisely, we would have produced a model whose true burn-in time was, unknowably, a figurative billion years.

Figure 15.8.2 shows how  $\lambda_1$  and  $\lambda_2$  distribute themselves during  $10^5$  steps after step 1000. This is the payoff of MCMC: We learn not just about most likely parameter values, but also details about how well the parameters are determined by this



**Figure 15.8.2.** After burn-in, the MCMC model is run for an additional  $10^5$  steps. Parameter values are saved every 10 steps, giving the histograms shown for parameters  $\lambda_1$  and  $\lambda_2$ . These histograms represent the inferred parameter values and their uncertainties. The model data were generated with parameters  $\lambda_1 = 3$  and  $\lambda_2 = 2$ . The inferred values for this particular sample of 1000 data points are seen to be about as accurate as should be expected from their uncertainties.

particular data set. We could also have shown any joint distribution of interest, or computed any average quantity, for example

$$\langle \lambda_1 \rangle = \frac{1}{n-k} \sum_{i=k}^{n-1} (\lambda_1)_i \quad (15.8.13)$$

Here  $k = 1000$  is the number of burn-in steps, which we reject;  $n = k + 10^5$  is the number of steps that are averaged; and  $(\lambda_1)_i$  denotes the value of  $\lambda_1$  at the  $i$ th step. Sums like (15.8.13) are called *ergodic averages*.

Some remarks should be made about equation (15.8.13): One is allowed to average all steps, even though successive steps are not independent samples of  $\pi(\mathbf{x})$ . One would also be allowed to include in the average only every  $m$ th step, where you choose  $m$  to be greater than some empirically observed correlation time in the Markov chain. The latter is sometimes recommended as a means of estimating the standard error of  $\langle \lambda_1 \rangle$ , just as in Monte Carlo integration. (Compare equations 7.7.1 and 7.7.2.) Warning: Doing this in the context of a finite data set is often associated with conceptual error. While it is true that as  $n \rightarrow \infty$ , equation (15.8.13) does converge to a precise value, it is *not* true that this value has anything to do with the actual (population) value of  $\lambda_1$ . Rather, it is just the best apparent (sample) value of  $\lambda_1$  for this particular data set. The relation of this apparent value to the actual value has nothing to do with the standard error of  $\langle \lambda_1 \rangle$ , but is instead indicated by the width of the distribution of all the  $(\lambda_1)_i$ 's.

Figure 15.8.2 illustrates this point well. By running the model for a long time, we could achieve beautifully precise distributions that have extremely well-converged means. But they would not be centered on the (here secretly known) true values 3 and 2. You should run an MCMC model only (i) long enough to be sure (or sure

enough) that there was sufficient burn-in, and (ii) long enough to characterize distributions well enough that the error in mean quantities of interest is reasonably small compared with the observed dispersion of those quantities in the Markov chain.

### 15.8.4 Other Aspects of MCMC

We have said little about how to determine the necessary length of burn-in, other than to advise you to *look at the output* (which is always a good idea). There is in fact a large literature on this subject [2-4]. A number of so-called *convergence diagnostic* tools have been developed. The problem is that, even when you use these tools, you really must *look at the output* anyway; so their added value is often not large. It is always a good idea to have the length of burn-in be *at least* 1 or 2% of the total length of your run, which will be determined by the accuracy that you need in estimating model parameters. Keep in mind that it is easy to construct scary examples of distributions  $\pi(\mathbf{x})$  full of false convergence traps. The more you know about your distribution, the better off you will be.

Multiple, independent Markov chains can be run to explore a single distribution  $\pi(\mathbf{x})$ . On a single processor the only reason to do this would be to meet some unusual need for independent samples of the distribution. However, on machines with multiple processors, this is a natural way of achieving efficient parallelization.

We have assumed that the number of dimensions in  $\mathbf{x}$  is fixed. It is possible to have models, however, in which the number of fitted parameters is itself a variable. These *variable dimension models* require special care in the design of proposal distributions that can step between different numbers of dimensions. See the paper by Phillips and Smith in [2] for an introduction.

### 15.8.5 Importance Sampling and MCMC

In §7.9 we noted that the error in Monte Carlo integration could be reduced by importance sampling, where we write (in the notation of this section)

$$I \equiv \int f(\mathbf{x}) d\mathbf{x} = \int \frac{f(\mathbf{x})}{p(\mathbf{x})} p(\mathbf{x}) d\mathbf{x} \quad (15.8.14)$$

for an aptly chosen  $p$ . We saw that the ideal  $p$  would (i) resemble  $f$  in functional form, cf. equation (7.9.6), and (ii) admit to a good method for sampling uniformly over  $p(\mathbf{x}) d\mathbf{x}$ .

You might think that MCMC provides a great general-purpose way to sample over any  $p$  and thus make importance sampling easy to implement in all cases. Unfortunately, no. The problem, once again (as for the Gibbs sampler), is the normalizing constant. MCMC's great virtue is that it samples over a distribution  $\pi(\mathbf{x})$  without requiring that it be normalized. If you ask what normalized probability distribution  $p(\mathbf{x})$  is actually being sampled over, it is of course

$$p(\mathbf{x}) = \frac{\pi(\mathbf{x})}{\int \pi(\mathbf{x}) d\mathbf{x}} \quad (15.8.15)$$

Equation (15.8.14) then becomes

$$I \equiv \int f(\mathbf{x}) d\mathbf{x} = \int \pi(\mathbf{x}) d\mathbf{x} \times \int \frac{f(\mathbf{x})}{\pi(\mathbf{x})} p(\mathbf{x}) d\mathbf{x} \quad (15.8.16)$$